

DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS DE SOFTWARE (SRS)

Sistema: CodeSurvive AI (Plugin-based Modernization Bridge)

Versão: 1.0

Data: 24 de Maio de 2026

Autor: Engenharia de Sistemas & Arquitetura de Soluções AI

Status: Aprovado para Desenvolvimento de MVP

1. INTRODUÇÃO E VISÃO GERAL

1.1 Resumo Executivo

O **CodeSurvive AI** é uma plataforma inovadora de modernização de software corporativo de missão crítica, simulada neste protótipo como uma extensão integrada para ambientes de desenvolvimento (IDEs) como VS Code, GitHub Codespaces e Google Apps Script.

Ao contrário de abordagens tradicionais que realizam tradução sintática literal de código ou tentam substituir o sistema legado de forma destrutiva, o CodeSurvive AI atua como uma **Ponte de Tradução Não-Destrutiva (Translation Bridge)**. O sistema preserva o progresso e a integridade do código legacional original, executando uma análise semântica linha a linha, identificando vulnerabilidades de arquitetura, gerando código moderno equivalente de forma isolada em um painel paralelo, e provisionando um ecossistema de infraestrutura híbrida (Mainframe Emulado em Nuvem para sistemas legados e Cloud IDE para sistemas modernos).

1.2 Missão do Produto

Empoderar equipes de engenharia de software na jornada de migração de arquiteturas obsoletas para ecossistemas modernos, reduzindo em mais de 45% o custo de desenvolvimento, eliminando o estresse operacional de engenharia reversa manual e mitigando riscos de regressão sistêmica com o uso de inteligência artificial de alta precisão.

2. O STORYTELLING CORPORATIVO: "A SEXTA-FEIRA DO APOCALIPSE OCULTO"

2.1 Os Personagens

- Lucas (Engenheiro de Software Sênior):** Especialista em arquiteturas modernas de microsserviços e nuvem, contratado para liderar a inovação tecnológica da empresa.
- Amanda (Diretora de Produto - CPO):** Responsável por garantir que novos recursos de negócios cheguem ao mercado antes da concorrência, sob forte pressão de metas trimestrais.
- O "Monolito" (Sistema Core de Faturamento):** Um sistema crítico legado que roda em background há mais de uma década. Não possui documentação, testes unitários ou arquitetura padronizada. Foi escrito em uma linguagem antiga por um engenheiro que não faz mais parte da organização.

2.2 O Incidente

Em uma quinta-feira, às 16h30, Amanda entra na sala de desenvolvimento com uma demanda de alta prioridade: a empresa fechou um contrato estratégico de R\$ 2 milhões com um player global, sob a condição de implementar uma regra de desconto dinâmico no sistema de faturamento corporativo em até 4 dias.

Lucas assume o desafio. Ao abrir o repositório do "Monolito", depara-se com um arquivo maciço contendo milhares de linhas de código procedural confuso, variáveis sem significado semântico aparente e um comentário assustador de 2016: `// NÃO ALTERAR ESTE BLOCO, QUEBRA O FLUXO DE CAIXA E NINGUÉM SABE O MOTIVO`.

Durante toda a sexta-feira, Lucas realiza uma verdadeira "arqueologia de software". Ferramentas de análise estática tradicionais apenas validam a sintaxe, mas são incapazes de explicar a lógica de negócio encapsulada ali. Pressionado pelo prazo de entrega, ele implementa a nova regra de desconto de forma isolada e realiza o deploy em homologação.

Às 19h30, os servidores de telemetria entram em alerta. A nova regra alterou inadvertidamente o fluxo de cálculo fiscal de transações internacionais antigas, emitindo notas com valores negativos. O código antigo possuía um acoplamento oculto e indocumentado. Lucas enfrenta um final de semana de trabalho ininterrupto e estressante sob o risco de cancelamento do contrato milionário.

2.3 O Diagnóstico do Problema

O incidente ilustra as três maiores dores do desenvolvimento de software corporativo atual:

1. **Falta de Contexto de Negócio:** Desenvolvedores gastam até 70% do tempo tentando deduzir a regra de negócio por trás de construções técnicas antigas.
2. **O Medo do Efeito Dominó (Blast Radius):** A ausência de mapeamento de dependências gera pavor de alterar uma linha de código e comprometer módulos periféricos.
3. **Ineficiência Financeira:** Engenheiros altamente qualificados são drenados para atuar como arqueólogos de código, em vez de focarem em inovação de produto.

3. ANÁLISE DO PROBLEMA E MODELAGEM DO DÉBITO TÉCNICO

Sistemas legados atuam como âncoras financeiras nas organizações. À medida que o código envelhece sem refatoração planejada, a manutenção acumula um "imposto silencioso" conhecido como Débito Técnico.

3.1 Equação de Acúmulo do Débito Técnico

A degradação do valor do ativo de software e o custo acumulado de sua manutenção podem ser modelados matematicamente através da seguinte equação de custo total de propriedade:

$$C_{total}(t) = C_{manutencao} \times (1 + r)^t + C_{oportunidade}(t)$$

Onde:

- $C_{total}(t)$ representa o Custo Total de Propriedade do sistema legado no tempo t .
- $C_{manutencao}$ representa o custo operacional inicial de manutenção do código base.
- r é a taxa anual de degradação e acúmulo de débito técnico (estimada pelo Gartner em aproximadamente 18% ao ano, ou $r = 0.18$).
- t é o tempo decorrido em anos desde a última modernização estrutural.

- $C_{oportunidade}(t)$ representa o custo de oportunidade decorrente do atraso no lançamento de novas features (*Time-to-Market*) causado pela lentidão de desenvolvimento no ecossistema obsoleto.

3.2 O Impacto em Números Reais

- **Trilhões em Jogo:** O custo global causado apenas por softwares mal escritos e sistemas legados ultrapassa **US\$ 2,41 trilhões anuais** apenas nos Estados Unidos.
- **Desperdício de Produtividade:** Desenvolvedores gastam em média **13.4 horas por semana** lidando com código espaguete e refatorações manuais de sistemas antigos. Isso equivale a aproximadamente **US\$ 85.000 perdidos anualmente** por desenvolvedor sênior alocado.
- **Bloqueio de Inovação:** Cerca de **72% do orçamento de TI** das empresas da Fortune 500 é drenado exclusivamente para manter sistemas antigos funcionando (*Run the Business*), deixando apenas 28% para o desenvolvimento de novos produtos (*Change the Business*).

4. REPOSITÓRIO DE REQUISITOS FUNCIONAIS (RF)

ID	Requisito Funcional	Descrição	Prioridade
RF-001	Interface Oculta/Retrátil	O painel da Inteligência Artificial deve iniciar completamente recolhido (escondido) e ser ativado exclusivamente mediante o clique no ícone da IA na Activity Bar lateral do VS Code simulado.	Crítica
RF-002	Seletores de Linguagem Dinâmicos	O plugin deve permitir a seleção dinâmica e condicional da linguagem legado (Origem) e da linguagem moderna (Destino) através de dropdowns interativos no painel expandido.	Crítica
RF-003	Análise de Paridade de Infraestrutura	Se o usuário selecionar como origem linguagens baseadas em hardware clássico (COBOL, Fortran, Assembly), o sistema deve ativar o módulo visual "Mainframe Emulado em Nuvem". Se selecionar linguagens web (PHP, Java, C), deve ativar o "Cloud IDE Core".	Alta
RF-004	Explicação de Código Semântica	O sistema deve gerar um painel com a tradução detalhada da lógica de funcionamento do código legado, focando em regras de negócio e não em sintaxe simples.	Alta
RF-005	Análise Linha a Linha de Falhas e Fragilidades	A IA deve listar de forma analítica e visual (cards vermelhos e amarelos) cada falha de segurança, vazamento de memória ou ponto de acoplamento rígido no código de origem.	Crítica
RF-006	Ponte de Refatoração Não-Destrutiva	O sistema deve gerar um código limpo e refatorado na linguagem de destino no painel da direita, mantendo o editor da esquerda intacto, sem risco de perda de progresso.	Crítica

RF-007	Marketplace de Novas Linguagens	A extensão deve disponibilizar um ecossistema de marketplace simulado onde o usuário pode instalar novos pacotes de tradução (ex: Go, Rust), que se integram aos seletores após o download.	Média
RF-008	Importação e Exportação Simétrica	Permitir o upload (importação) e download (exportação) independente tanto para os arquivos legados de entrada quanto para o código moderno e limpo gerado pela IA.	Alta

5. REQUISITOS NÃO-FUNCIONAIS (RNF)

ID	Requisito Não-Funcional	Critério de Aceitação / Métrica	Prioridade
RNF-001	Latência de Resposta	O processamento da IA e a transição visual do Split Screen não devem exceder 5 segundos ao utilizar mocks internos ou 15 segundos em chamadas reais de API.	Alta
RNF-002	Segurança e Privacidade	O sistema deve operar sob políticas de Zero Data Retention corporativa. Nenhuma linha de código inserida pode ser usada para treinamento de modelos de linguagem públicos.	Crítica
RNF-003	Design System e Fidelidade	A interface do protótipo deve simular rigorosamente o tema "Dark+ Modern" do Visual Studio Code, preservando tipografia, paletas de cores Slate, bordas de 1px e espaçamentos consistentes.	Alta
RNF-004	Emulação de Mainframe	O simulador de Mainframe na nuvem deve apresentar telemetria reativa realista, incluindo paridade de conjunto de caracteres (EBCDIC para ASCII) e uso simulado de registradores.	Média
RNF-005	Robustez e Tratamento de Erro	O sistema deve impedir o acionamento do pipeline se o editor de código de origem estiver vazio, emitindo um toast nativo estilizado na parte inferior da tela, sem uso de popups destrutivos do sistema operacional.	Alta

6. ESPECIFICAÇÃO DE INTERFACE (UI/UX) E FLUXOS INTERATIVOS

A experiência do usuário (UX) foi projetada para emular as ferramentas de elite utilizadas por engenheiros de software no dia a dia, minimizando a curva de aprendizado.

[VSC Header]			CodeSurvive AI MVP v1.0			[Gemini Key: *****]		
A	IA	EDITOR LEGADO (Original)				WORKSPACE IA		
C	Side							
T	bar	> legacy_file.cob				[Explicação]		
I								
V	Legado	01	IDENTIFICATION DIVISION.			[Falhas Técnicas]		
I	[COB]	02	PROGRAM-ID. BILLING.					
T	->	03	...			[Refatoração]		
Y	Dest.							

		[PY]			> output_clean.py	
	B				01 import json	
	A		[RUN]		02 def bill()...	
	R					

	[VSC Status Bar]	INFRA: MAINFRAME EMULADO [ONLINE]	
--	------------------	-----------------------------------	--

6.1 Comportamento do Painel Retrátil

- **Estado Fechado (Default):** O painel do Workspace da IA (Direita) e a Sidebar de controle estão invisíveis. O editor de código legado do painel esquerdo ocupa 100% da largura disponível, proporcionando uma área de digitação livre de distrações.
- **Abertura do Painel:** Ao clicar no ícone do CodeSurvive AI na Activity Bar, a Sidebar de configurações expande de *0px* para *320px*.
- **Ativação do Pipeline:** Ao acionar o botão de análise, o painel direito de saída da IA se projeta dividindo a tela central em um layout simétrico de 50/50 (Split-screen), ideal para comparação de código e tomadas de decisão arquiteturais.

6.2 O Pipeline Metodológico de Modernização (As Três Abas de IA)

Em vez dos termos abstratos de engenharia "Passo 1, 2 e 3", a solução foca nos objetivos diretos do desenvolvedor através de abas semânticas:

1. **Aba "Explicação do Código":** Realiza a engenharia reversa conceitual, descrevendo o fluxo lógico e as regras comerciais enterradas em rotinas obsoletas.
2. **Aba "Falhas Técnicas e Fragilidades":** Atua como o cérebro crítico de controle de qualidade, mapeando riscos de injeção de dados, estouro de buffer, acoplamento insustentável de dependências globais e trechos de código mortos.
3. **Aba "Refatoração":** O produto final da ponte de tradução. Um código novo, elegante, portátil, performático e estruturado sob os melhores paradigmas da linguagem de destino selecionada pelo usuário.

7. INTEGRAÇÃO DE INFRAESTRUTURA DINÂMICA (HYBRID CLOUD RUNTIME)

Um dos grandes diferenciais do CodeSurvive AI é a inteligência de sensibilidade ao contexto de hardware.

- **Linguagens Clássicas de Mainframe (COBOL, Fortran, Assembly):** Exigem infraestrutura virtualizada. O sistema conecta o código ao módulo **Mainframe Emulado em Nuvem**. O painel de telemetria exibe o status de emulação ativado, simulando a execução paralela das instruções para validar se o novo código traduzido entrega exatamente o mesmo resultado numérico que o hardware clássico geraria.
- **Linguagens Corporativas Modernas/Web (PHP, Java, C):** Não necessitam de hardware virtualizado de mainframe. A infraestrutura muda automaticamente para o **Cloud IDE Core**, operando sob contêineres virtuais ágeis e leves de desenvolvimento, economizando recursos de processamento em nuvem.

8. MODELO DE ESCALABILIDADE COM MARKETPLACE

A plataforma é projetada para ser extensível através de um ecossistema descentralizado de pacotes de tradução (Language Translation Packs).

- O desenvolvedor pode acessar a aba de **Marketplace** interna do plugin.
- Ao encontrar uma linguagem que deseja adicionar à ferramenta (ex: tradutor para a linguagem Rust), o usuário realiza o download do pacote diretamente pela interface.
- O banco de dados local da inteligência artificial é atualizado dinamicamente em tempo real, injetando as novas regras de gramática e compilação nos dropdowns de destino, expandindo o ciclo de vida comercial e operacional da plataforma.

9. MATRIZ DE RISCOS DO PRODUTO

Risco 1: Alucinação em Regras Fiscais Críticas.

- *Impacto:* Crítico.
- *Mitigação:* A aba de explicações e falhas é obrigada a exibir as linhas de origem correspondentes lado a lado com o código antigo. Além disso, a emulação de mainframe executa validações funcionais de ponta a ponta.

Risco 2: Resistência da Equipe de Engenharia Legada.

- *Impacto:* Médio-Alto.
- *Mitigação:* A abordagem não destrutiva do CodeSurvive AI garante que o desenvolvedor tenha controle total sobre a migração. O código original nunca é alterado sem autorização explícita; a ferramenta atua como um assistente consultivo